ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas

Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N.º 1

Práctica 1 - Consolidación de reportes de ventas

Estudiante: Medina Arcos Juan Sebastián - https://github.com/JuanMedina02

Docente: Christian Suárez

Período académico: 2026-B

Fecha de entrega: 12/10/2026

Repositorio : `https://github.com/JuanMedina02/IDSD422-lab1-lectura-datos`

1. Introducción

En el presente documento se realizó el estudio de un caso hipotético de una cadena comercial que ha proporcionado sus archivos de ventas en los cuales se detallan la sucursal, ventas de la misma y clientes. Los archivos de datos proporcionados se reciben fragmentados en tres formatos de archivo distintos: CSV, JSON y XLSX.

Para nuestro análisis de las diferentes métricas se consideró una buena opción el uso de los lenguajes de programación Python y R. En el caso de Python debido a su biblioteca pandas hecha específicamente para el análisis de datos y R al ser un lenguaje hecho con fines netamente estadísticos.

2. Objetivos

Realizar una lectura de archivos de datos en los diferentes formatos CSV, JSON y Excel utilizando los lenguajes de programación Python y R.

Calcular valores medios utilizando la información de los archivos de datos.

Determinar patrones de venta en base a los datos proporcionados.

3. Desarrollo de la práctica

Para la realización del componente en Python de la presente práctica se trabajó sobre un entorno de desarrollo controlado utilizando Miniconda, para la gestión de paquetes, control de versiones y entornos virtuales . A través del gestor de paquetes `conda`, se creó un entorno específico donde se utilizó la versión de de Python 3.14.6. Mientras que para la componente en el lenguaje R se trabajo utilizando la versión *2026.09.0+174* de R. Los scrips de Python se corrieron directamente en el terminal de Ubuntu mientras que los de R en Rstudio.

En las secciones 3.1 y 3.2 presentadas a continuación se muestra como se realizaron las tareas de lectura de datos y análisis estadístico estadístico exploratorio de los archivos de datos en formato CSV, JSON y XLSX usando Python y R.

3.1. Lectura de datos en Python

Para realizar la lectura de los archivos de datos CSV, JSON y XLSX utilizando Python se utilizó la biblioteca *pandas* la cual destaca por al utilización de DataFrames los cuales son una forma rápida y eficiente para la manipulación de datos con indexación integrada[2]

3.1.1. Lectura de datos de un archivo CSV

Para la lectura del archivo CSV, se utilizó la biblioteca `pandas`. Mediante la función `read_csv()`, se cargó el conjunto de datos desde el directorio local y se almacenó en una estructura DataFrame. Posteriormente, se realizó una exploración inicial utilizando el método `head()` para inspeccionar las primeras filas, junto con los atributos `shape` y `dtypes` para determinar las dimensiones totales de la tabla y los tipos de variables de cada columna, respectivamente.

El código completo implementado en Python para realizar esta importación y exploración se encuentra detallado en la sección de Anexos (Código 1), mientras que los resultados obtenidos al ejecutar el documento se visualizan en la Figura 3.

3.1.2. Lectura de datos de un archivo JSON

El procesamiento del formato JSON requirió la importación conjunta de las bibliotecas `json` y `pandas`. Inicialmente, se extrajo la información abriendo el archivo local con la función `json.load()`, para luego normalizarla y convertirla en un DataFrame utilizando la función `json_normalize()`. Una vez estructurados los datos, se aplicaron exactamente los mismos métodos de inspección inicial descritos para el archivo CSV.

El script desarrollado para llevar a cabo este proceso puede consultarse íntegramente en la sección de Anexos (Código 2) y la salida generada por su ejecución se presenta en la figura correspondiente a la lectura JSON.

3.1.3. Lectura de datos de un archivo XLSX

Para leer este formato, fue indispensable instalar previamente la biblioteca `openpyxl`, la cual actúa como el motor que permite a `pandas` interpretar la estructura de las hojas de cálculo y extraer los datos [1]. Tras esta configuración, se empleó la función `read_excel()` para cargar la información en un DataFrame, finalizando con los mismos pasos de exploración de datos descritos en los apartados anteriores.

El fragmento de código empleado para esta tarea está disponible en la sección de Anexos (Código 3), y los resultados en consola de esta extracción de datos se ilustran en su respectiva figura de resultados.

3.2. Inspección de datos en Python

3.2.1. Generación de una columna derivada

Se generó una columna derivada con nombre *ticket* la cual es el resultado de dividir el número de ventas sobre el número de clientes para cada fila de nuestro DataFrame. Para lo cual se implemento la siguiente instrucción.

```
pd_ventas_CSV['Ticket'] = pd_ventas_CSV['ventas']/pd_ventas_CSV['clientes']
```

3.2.2. Venta promedio por sucursal

Para encontrar las ventas promedio por sucursal se utilizó la la función `groupby` de pandas las cual permite agrupar agrupar los datos de un DataFrame como una función de los valores de una o más columnas categóricas [2]. En este caso se agruparón los datos de la columna *ventas* en función de las sucursales de la columna *sucursal* y esto se almaceno en la variable `sucursales`. A continuación de calculo el promedio de la variable `sucursales` usando la función `mean()`.

```
sucursales = pd_ventas_CSV.groupby('sucursal')['ventas']
print(sucursales.mean())
```

3.2.3. Tickets Promedio

Se calculó el número promedio de tikets vendidos como el resultado de las ventas totales sobre el número de clientes totales de todas las sucursales.Se calcularon el número total de ventas y clientes de nuestro DataFrame usando la función `.sum()` y se almacenaron en las variables `ventas_totales` y `clientes_totales` respectivamente.A continuación se calculó y almaceno el valor promedio de tickets en una variable llamada `prom_ticket` dividiento las variables `ventas_totales` sobre `clientes_totales`. Y finalmente se imprimio el valor promedio de tickets.

```
ventas_totales = pd_ventas_CSV['ventas'].sum()
clientes_totales =pd_ventas_CSV['clientes'].sum()
prom_ticket = ventas_totales / clientes_totales
print('=', prom_ticket)
```

3.2.4. Sucursales con mayores y menores ventas

Se comenzo encontrando las filas para los cuales los valores de la columna **ventas** presentaba los valores máximos y mínimos usandó la función `.idmax()` e `.idmin()` las cuales permiten encontrar el índice de una fila o columna donde se encuentra el valor máximo y mínimo respectivamente en una Serie o un DataFrame[3] [4]. Estos valores se guardaron en las varibales `fila_max` y `fila_main`. A continuación, se utilizo la función `.loc()` para extraer la sucursal con mayor y menor ventas en la columna *sucursal* usando los indices proporcionados por las funciones `fila_max` y `fila_main`.Los resultados obtenidos fueron guardados en las variables `surcursal_max` y **sucursal_min** respectivamente.

```
fila_max = pd_ventas_CSV['ventas'].idxmax()
fila_min = pd_ventas_CSV['ventas'].idxmax()

sucursal_max = pd_ventas_CSV.loc[fila_max, 'sucursal']
sucursal_min = pd_ventas_CSV.loc[fila_max, 'sucursal']
```

3.2.5. Región con mayor venta total

Usando la función `groupby` se agruparon los datos de la columna *ventas* en función de las regiones proporcionadas por la columna *region*y usando la función `.sum` se sumaron los valores correspondientes de cada región. Finalmente toda esa información se almaceno en una variable llamada `regiones`. A continuación se determinó la fila a la región con mayor cantidad de ventas usando la función `.idmax`

```
regiones = pd_ventas_CSV.groupby('region')['ventas'].sum()
region_max = regiones.idxmax()
value_region_max = regiones.max()
```

El código completo implementado para la realización de todas las actividades de mostradas en la sección **3.2** se lo puede encontrar en la sección de Anexos en el Código 4. Y los resultados obtenidos en la Figura 7.

3.3. Lectura de datos con R

Para la realización de la lectura de archivos con formato CSV, JSON y XLSX usando R, se utilizó el programa RStudio para la ejecución del código y la visualización de resultados.

3.3.1. Lectura de datos de un archivo CSV

Para leer el archivo CSV se instaló y utilizó la biblioteca `readr`. Mediante la función `read_csv()` se cargaron los datos en un *tibble* (una versión moderna de data frame). Posteriormente, se emplearon las funciones base `head()`, `dim()` y `str()` para visualizar las primeras filas, obtener las dimensiones totales y determinar el tipo de variable de cada columna, respectivamente.

3.3.2. Lectura de datos de un archivo JSON

Se empleó la biblioteca `jsonlite` para importar el archivo JSON. El uso de la función `fromJSON()` permitió extraer y convertir automáticamente la información a una estructura de *data frame*. Las dimensiones y tipos de datos se verificaron con las mismas funciones exploratorias mencionadas en la sección anterior.

3.3.3. Lectura de datos de un archivo XLSX

La extracción de datos del libro de Excel se llevó a cabo con la biblioteca `readxl`. Utilizando la función `read_excel()` y definiendo explícitamente la hoja requerida mediante el argumento `sheet="ventas"`, se cargó la información de manera exitosa para su posterior exploración estructural con la función `str()`.

Los códigos implementados para las secciones **3.3.1**, **3.3.2** y **3.3.3** se pueden encontrar en la sección de Anexos en los Códigos 5, 6 y 7 respectivamente.

3.4. Inspección de datos en R

En R se realizó exactamente el mismo análisis exploratorio que se realizó en Python. La lógica de las funciones utilizadas son muy similares a las usadas en Python. El código implementado se encuentra comentado y detallado de manera muy explícita para facilitar su entendimiento en la sección de Anexos en el Código 8 y sus resultados en la Figura 8.

Preguntas

¿Cuántas filas y columnas tiene la tabla en cada uno de los tres formatos? ¿Fue igual en los tres? ¿Por qué?

Las filas y columnas en los tres archivos de datos fueron 5 filas y 3 columnas, debido a que los datos unicámente se encontraban guardados en formatos diferentes.

¿Qué tipo de datos se infirió para ventas y clientes? ¿Qué indicaría que salieran como texto?

Los datos obtenidos para las columans de *ventas* y **clientes** fueron del tipo *int* debido a que se media cantidad, si fuesen texto indicaría que las tablas fueron llenada de forma erronea.

En Python, ¿qué estructura devuelve json.load y por qué hay que aplanarla?

La función `json.load()` nos devuelve un diccionario de Python. Es necesario .ªplanar.º normalizar esta información porque los archivos JSON suelen tener una estructura anidada o

jerárquica. Al aplanarla, logramos transformar esos datos en una tabla plana de filas y columnas.
¿Para qué sirve indicar sheet_name/sheet al leer Excel?
Sirve para saber de qué hoja exacta del libro de Excel queremos extraer los datos. Dado que los archivos XLSX pueden contener múltiples hojas de cálculo.

¿Qué diferencia ves entre leer en Python y leer en R (conceptos, no sintaxis)?
La principal diferencia es que R es un lenguaje de programación diseñado específicamente para realizar estadística con sus propias funciones. Mientras que Python es un lenguaje de propósito general, por lo que para leer y manipular datos dependemos de bibliotecas externas como `pandas` usando un enfoque orientado a objetos

4. ¿Qué aprendí con esta práctica?

Se aprendieron los procedimientos para realizar la importación y lectura de archivos de datos con diferentes formatos como CSV, XLSX y JSON a Python y R.

Uso de la biblioteca *pandas* de Python para la lectura y análisis exploratorio de datos

Uso de diferentes bibliotecas y funciones específicas en R y Python para realizar análisis exploratorios de archivos de datos.

5. Conclusiones

Los resultados obtenidos al ejecutar los códigos de lectura mostraron que independientemente del formato del archivo de lectura los resultados son los mismos, no se observaron variaciones números ni de cambios en los tipos de variables extraídas de los archivos.

Los lenguajes Python y R son igualmente viables para realizar las actividades de lectura y análisis exploratorio de datos puesto que ambos poseen bibliotecas ya específicamente diseñadas para estas actividades.

Referencias

[1] Eric Gazoni and Charlie Clark. openpyxl documentation. `https://openpyxl.readthedocs.io/en/stable/`, 2026. Accedido: 12 de octubre de 2026.

[2] The pandas development team. About pandas - pandas documentation. `https://pandas.pydata.org/about/index.html`, 2026. Accedido: 12 de octubre de 2026.

[3] The pandas development team. pandas.dataframe.idxmax documentation. `https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.idxmax.html`, 2026. Accedido: 19 de octubre de 2026.

[4] The pandas development team. pandas.dataframe.idxmin documentation. `https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.idxmin.html`, 2026. Accedido: 19 de octubre de 2026.

Anexos

Código 1: Código completo utilizado para la lectura del archivo de datos ventas.csv

```
import pandas as pd    #Importaci n de bibliotecas

print('Lectura de archivos tipo CSV')
pd_ventas_CSV=pd.read_csv("../data/ventas.csv")    #Lectura CSV

print('Impresi n de las primeras filas del archivo CSV')
print(pd_ventas_CSV.head())
print('\nDimensiones: \n',pd_ventas_CSV.shape)         #Dimensiones del
    DataFrame
print('\nTipos por columna: \n', pd_ventas_CSV.dtypes) #Variables de cada
    columna
```

Código 2: Código completo utilizado para la lectura del archivo de datos ventas.json

```
import json
import pandas as pd     # Importaci n de bibliotecas
with open("data/ventas.json", encoding="utf-8") as f:  #Abrimos el archivo JSON
    datos = json.load(f)
pd_ventas = pd.json_normalize(datos)      # Normalizaci n y conversion a
    DataFrame
#============================================================
print('Impresi n de las primeras filas del archivo JSON')
print(pd_ventas.head())

print('\nDimensiones: ',pd_ventas.shape)              # Diemnsiones del
    DataFrame
print('\nTipos por columna: ')
print(pd_ventas.dtypes)
```

Código 3: Código completo utilizado para la lectura del archivo de datos ventas.xlsx

```

import pandas as pd

pd_ventas = pd.read_excel("data/ventas.xlsx", sheet_name='ventas') # Leemos el
    archivo Excel con pandas

#============================================================
print('Impresi n de las primeras filas del archivo XLSX')

print(pd_ventas.head())

print('Dimensiones: \n', pd_ventas.shape)
print('\nTipos por columna: ')
print(pd_ventas.dtypes)
```

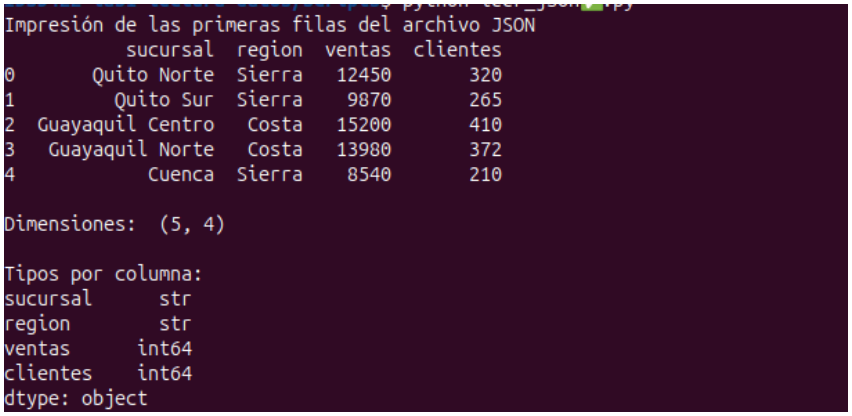


Figura 1: Resultados obtenidos obtenidos al realizar la lectura del archivo ventas.csv utulizando el código 1

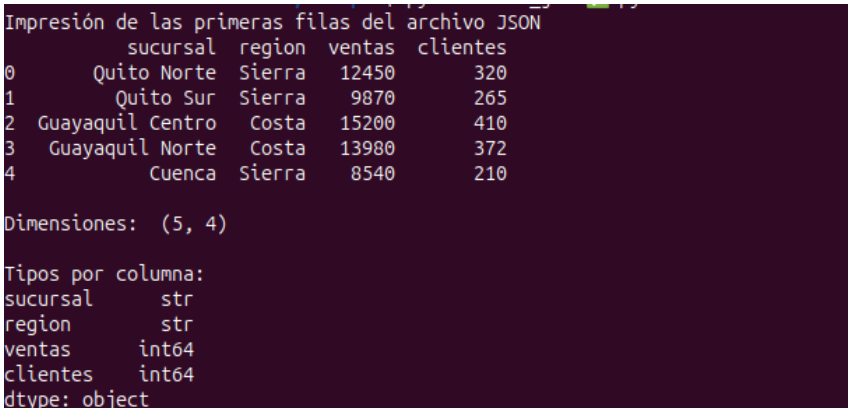


Figura 2: Resultados obtenidos obtenidos al realizar la lectura del archivo ventas.json utulizando el código 2

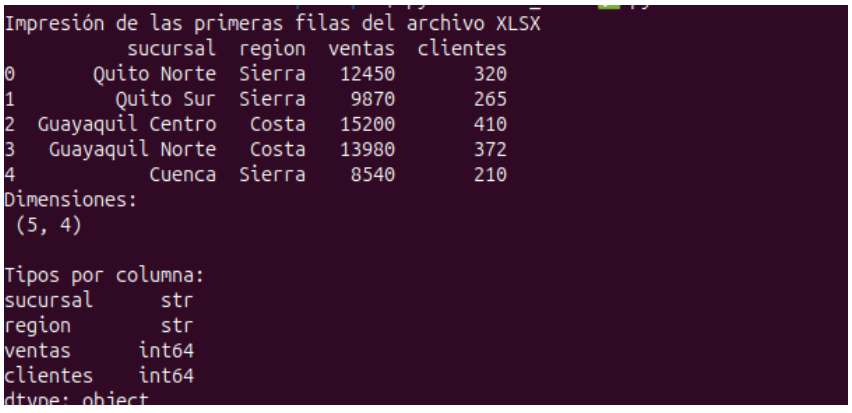


Figura 3: Resultados obtenidos obtenidos al realizar la lectura del archivo ventas.xlsx utulizando el código 3

Código 4: Código completo utilizado para las actividades de inspección del archivo de datos ventas.csv

```
import pandas as pd

print('Lectura de archivos tipo CSV')
pd_ventas_CSV=pd.read_csv("../data/ventas.csv")    #Lectura del archivo ventas.
    csv

print('\nTipos por columna: \n', pd_ventas_CSV.dtypes) # Impresi n de
    dataTypes

#=====================================================================
# Generaci n de la columna "Ticket" = ventas/clientes
pd_ventas_CSV['Ticket'] = pd_ventas_CSV['ventas']/pd_ventas_CSV['clientes']
print('\nDataFrame con nueva columna ',pd_ventas_CSV.head())  # Impresi n

#=====================================================================
print('\nVenta promedio por sucursal')
sucursales = pd_ventas_CSV.groupby('sucursal')['ventas']
print(sucursales.mean())

#=====================================================================
print('\nTicket Promedio (ventas totales/clientes totales) ')
ventas_totales = pd_ventas_CSV['ventas'].sum()
clientes_totales =pd_ventas_CSV['clientes'].sum()
prom_ticket = ventas_totales / clientes_totales
print('=', prom_ticket)

#=====================================================================
print('\n Sucursal con mayor y menor venta')
fila_max = pd_ventas_CSV['ventas'].idxmax() # Fila con mayores ventas
fila_min = pd_ventas_CSV['ventas'].idxmax() #Fila con menores ventas

sucursal_max = pd_ventas_CSV.loc[fila_max, 'sucursal']
sucursal_min = pd_ventas_CSV.loc[fila_max, 'sucursal']

print('Sucursal con mayores ventas: ', sucursal_max)
print('Sucursal con menores ventas: ', sucursal_min)

#=====================================================================
print('\nRegion con mayor venta total ')
regiones = pd_ventas_CSV.groupby('region')['ventas'].sum() #Agrupar regiones
region_max = regiones.idxmax()
value_region_max = regiones.max()
print('Region con mayores ventas: ', region_max)
print('Con ventas totales de: ', value_region_max)
```

```
Tipos por columna:
 sucursal      str
region        str
ventas      int64
clientes    int64
dtype: object

DataFrame con nueva columna            sucursal  region  ventas  clientes     T
icket
0       Quito Norte  Sierra   12450       320  38.906250
1         Quito Sur  Sierra    9870       265  37.245283
2  Guayaquil Centro   Costa   15200       410  37.073171
3   Guayaquil Norte   Costa   13980       372  37.580645
4            Cuenca  Sierra    8540       210  40.666667

Venta promedio por sucursal
sucursal
Cuenca               8540.0
Guayaquil Centro    15200.0
Guayaquil Norte     13980.0
Quito Norte         12450.0
Quito Sur            9870.0
Name: ventas, dtype: float64

Ticket Promedio (ventas totales/clientes totales)
= 38.0722891566265

 Sucursal con mayor y menor venta
Sucursal con mayores ventas:  Guayaquil Centro
Sucursal con menores ventas:  Cuenca

Región con mayor venta total
Región con mayores ventas:  Sierra
Con ventas totales de:  30860
```

Figura 4: Resultados obtenidos obtenidos al realizar la inspección del archivo de datos ventas.csv

Código 5: Código utilizado para la lectura del archivo de datos ventas.csv en R

```r
install.packages('readr')
library(readr)
ventas_csv <- read_csv("../data/ventas.csv")

cat("\nImpresi n de las primeras filas del archivo CSV:\n")
print(head(ventas_csv))

cat("\nDimensiones (filas, columnas):\n")
print(dim(ventas_csv))

cat("\nEstructura y tipos por columna:\n")
str(ventas_csv)
```

Código 6: Código utilizado para la lectura del archivo de datos ventas.json en R

```r
install.packages('jsonlite')
library(jsonlite)

cat("Lectura de archivos tipo JSON\n")
# fromJSON lee el archivo y lo convierte autom ticamente en un data.frame
ventas_json <- fromJSON("../data/ventas.json")

cat("\nImpresi n de las primeras filas del archivo JSON:\n")
print(head(ventas_json))

cat("\nDimensiones (filas, columnas):\n")
print(dim(ventas_json))

cat("\nEstructura y tipos por columna:\n")
str(ventas_json)
```

Código 7: Código utilizado para la lectura del archivo de datos ventas.xlsx en R

```r
install.packages('readxl')
library(readxl)

cat("Lectura de archivos tipo XLSX\n")
# read_excel nos permite especificar la hoja a leer
ventas_xlsx <- read_excel("../data/ventas.xlsx", sheet = "ventas")

cat("\nImpresi n de las primeras filas del archivo XLSX:\n")
print(head(ventas_xlsx))

cat("\nDimensiones (filas, columnas):\n")
print(dim(ventas_xlsx))

cat("\nEstructura y tipos por columna:\n")
str(ventas_xlsx)
```

```
Impresión de las primeras filas del archivo CSV:
# A tibble: 5 × 4
  sucursal         region ventas clientes
  <chr>            <chr>   <dbl>    <dbl>
1 Quito Norte      Sierra  12450      320
2 Quito Sur        Sierra   9870      265
3 Guayaquil Centro Costa   15200      410
4 Guayaquil Norte  Costa   13980      372
5 Cuenca           Sierra   8540      210

Dimensiones (filas, columnas):
[1] 5 4

Estructura y tipos por columna:
spc_tbl_ [5 × 4] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ sucursal: chr [1:5] "Quito Norte" "Quito Sur" "Guayaquil Centro" "Guayaquil Norte" ...
 $ region  : chr [1:5] "Sierra" "Sierra" "Costa" "Costa" ...
 $ ventas  : num [1:5] 12450 9870 15200 13980 8540
 $ clientes: num [1:5] 320 265 410 372 210
 - attr(*, "spec")=
  .. cols(
  ..   sucursal = col_character(),
  ..   region = col_character(),
  ..   ventas = col_double(),
  ..   clientes = col_double()
  .. )
 - attr(*, "problems")=<externalptr>
```

Figura 5: Resultados obtenidos realizar la lectura del archivo ventas.csv usando R al ejecutar el código5

```
Lectura de archivos tipo JSON

Impresión de las primeras filas del archivo JSON:
          sucursal region ventas clientes
1      Quito Norte Sierra  12450      320
2        Quito Sur Sierra   9870      265
3 Guayaquil Centro  Costa  15200      410
4  Guayaquil Norte  Costa  13980      372
5           Cuenca Sierra   8540      210

Dimensiones (filas, columnas):
[1] 5 4

Estructura y tipos por columna:
'data.frame':   5 obs. of  4 variables:
 $ sucursal: chr  "Quito Norte" "Quito Sur" "Guayaquil Centro" "Guayaquil Norte" ...
 $ region  : chr  "Sierra" "Sierra" "Costa" "Costa" ...
 $ ventas  : int  12450 9870 15200 13980 8540
 $ clientes: int  320 265 410 372 210
>
```

Figura 6: Resultados obtenidos realizar la lectura del archivo ventas.json usando R al ejecutar el código6

```
Lectura de archivos tipo XLSX

Impresión de las primeras filas del archivo XLSX:
# A tibble: 5 × 4
  sucursal         region ventas clientes
  <chr>            <chr>   <dbl>    <dbl>
1 Quito Norte      Sierra  12450      320
2 Quito Sur        Sierra   9870      265
3 Guayaquil Centro Costa   15200      410
4 Guayaquil Norte  Costa   13980      372
5 Cuenca           Sierra   8540      210

Dimensiones (filas, columnas):
[1] 5 4

Estructura y tipos por columna:
tibble [5 × 4] (S3: tbl_df/tbl/data.frame)
 $ sucursal: chr [1:5] "Quito Norte" "Quito Sur" "Guayaquil Centro" "Guayaquil Norte" ...
 $ region  : chr [1:5] "Sierra" "Sierra" "Costa" "Costa" ...
 $ ventas  : num [1:5] 12450 9870 15200 13980 8540
 $ clientes: num [1:5] 320 265 410 372 210
```

Figura 7: Resultados obtenidos realizar la lectura del archivo ventas.xlsx usando R al ejecutar el código7

Código 8: Código completo modificado utilizado para las actividades de inspección del archivo de datos ventas.csv en R

```r
library(readr)  # Importacion de librerias

cat("=========================================\n")
cat(" Carga de datos y estructura\n")
cat("=========================================\n")
ventas_csv <- read_csv("../data/ventas.csv", show_col_types = FALSE)

cat ('Estructura de los datos')
str(ventas_csv)

cat("Generaci n de una columna derivada 'ticket'\n")
cat("=========================================\n")
ventas_csv$ticket <- ventas_csv$ventas / ventas_csv$clientes
print(head(ventas_csv))

cat("Venta Promedio por sucursal\n")
cat("=========================================\n")
# usamos la funci n aggregate
promedio_sucursal <- aggregate(ventas ~ sucursal, data = ventas_csv, FUN = mean
    )
print(promedio_sucursal)

cat("\nTicket Promedio (ventas totales / clientes totales):\n")
cat("=========================================\n")
ventas_totales <- sum(ventas_csv$ventas)
clientes_totales <- sum(ventas_csv$clientes)
prom_ticket <- ventas_totales / clientes_totales
cat("=", prom_ticket, "\n")

cat("\n Sucursal con mayores y menores ventas\n")
cat("=========================================\n")
# which.max y which.min son el equivalente directo de idxmax() e idxmin() en
    pandas
fila_max <- which.max(ventas_csv$ventas)
fila_min <- which.min(ventas_csv$ventas)

sucursal_max <- ventas_csv$sucursal[fila_max]
sucursal_min <- ventas_csv$sucursal[fila_min]

cat("Sucursal con mayores ventas: ", sucursal_max, "\n")
cat("Sucursal con menores ventas: ", sucursal_min, "\n")

cat("\nRegi n con mayor venta total:\n")
cat("=========================================\n")
ventas_region <- aggregate(ventas ~ region, data = ventas_csv, FUN = sum)
fila_region_max <- which.max(ventas_region$ventas)

region_max <- ventas_region$region[fila_region_max]
value_region_max <- ventas_region$ventas[fila_region_max]

cat("Regi n con mayores ventas: ", region_max, "\n")
cat("Con ventas totales de: ", value_region_max, "\n")
```

```
Generación de una columna derivada 'ticket'
========================================
# A tibble: 5 × 5
  sucursal         region ventas clientes ticket
  <chr>            <chr>   <dbl>    <dbl>  <dbl>
1 Quito Norte      Sierra  12450      320   38.9
2 Quito Sur        Sierra   9870      265   37.2
3 Guayaquil Centro Costa   15200      410   37.1
4 Guayaquil Norte  Costa   13980      372   37.6
5 Cuenca           Sierra   8540      210   40.7
Venta Promedio por sucursal
========================================
          sucursal ventas
1           Cuenca   8540
2 Guayaquil Centro  15200
3  Guayaquil Norte  13980
4      Quito Norte  12450
5        Quito Sur   9870

Ticket Promedio (ventas totales / clientes totales):
========================================
= 38.07229

 Sucursal con mayores y menores ventas
========================================
Sucursal con mayores ventas:  Guayaquil Centro
Sucursal con menores ventas:  Cuenca

Región con mayor venta total:
========================================
Región con mayores ventas:  Sierra
Con ventas totales de:  30860
```

Figura 8: Resultados obtenidos realizar la inspección del archivo ventas.csv usando R al ejecutar el código 8